Project 3 – Main Simulator Documentation

Michael Mejalli

Equipment Classes

(Bus, Transformer, Transmission Line, Conductor, Bundle, Geometry)

Overview

The equipment classes and subclasses allow a user to propagate a circuit object and actually perform meaningful analyses. These classes include the Bus class, Transformer class, and Transmission Line class. Further subclasses include the Conductor class, Bundle class, and Geometry class.

Bus Class

Attributes:

- **name** (str) – Name of the bus.
- **base_kv** (float) – The base voltage level of the bus in kilovolts.
- **index** (int) – A unique index assigned to the bus based on creation order.
- **vpu** (float) – Per-unit voltage magnitude of the bus (default is 1.0).
- **delta** (float) – Voltage phase angle in degrees (default is 0).
- **bus_type** (str) – Type of the bus. Must be one of: "Slack_Bus", "PQ_Bus", or "PV_Bus".
- **generator** (Generator) – A placeholder generator object assigned by default. Can be replaced with a valid generator later.
- **load** (Load) – A placeholder load object assigned by default. Can be updated with actual load data.

Methods:

- **__init__**(self, name: str, base_kv: float, vpu: float = 1, delta: float = 0, bus_type: str = "PQ_Bus") – Constructor that initializes a Bus object. Assigns a unique index to each bus. Sets voltage magnitude (vpu), angle (delta), and bus type (PQ_Bus, PV_Bus, or Slack_Bus). Initializes default placeholder Load and Generator objects. Invalid bus types are rejected with a message.

- **__str__**(self) **–** Returns a formatted string representation of the bus, including the bus name, index, and base voltage.

Transformer Class

Attributes:

- **name** (str) – Name of the transformer.
- **bus1** (Bus) – First connected Bus object.
- **bus2** (Bus) – Second connected Bus object.
- **power_rating** (float) – Power rating of the transformer in MVA.
- **impedance_percent** (float) – Impedance of the transformer as a percentage of base impedance.
- **x_over_r_ratio** (float) – X/R ratio of the transformer.
- **impedance** (complex) – Per unit complex impedance calculated based on given parameters.
- **admittance** (complex) – Admittance calculated as reciprocal of the impedance.
- **y_prim_mat** (np.array) – Primitive admittance matrix of the transformer.

Methods:

- **__init__**(self, name: str, bus1: Bus, bus2: Bus, power_rating: float, impedance_percent: float, x_over_r_ratio: float) – Constructor that initializes the Transformer object and computes its impedance and admittance.
- **calculate_impedance**(self) – Computes the complex per unit impedance based on base values and X/R ratio.
- **calculate_admittance**(self) – Returns the admittance as the reciprocal of the transformer's impedance.
- **yprim**(self) – Computes and returns the 2x2 primitive admittance matrix for the transformer.
- **Rpu_Xpu**(self) – Returns the per unit resistance (Rpu) and reactance (Xpu) calculated from the X/R ratio.
- **__str__**(self) – Returns a formatted string with key transformer information for display or debugging.

Transmission Line Class

- **name** (str) **-** Name of the transmission line.
- **bus1** (Bus) **-** First bus object connected to the line.
- **bus2** (Bus) **-** Second bus object connected to the line.
- **bundle** (Bundle) **-** Bundle object representing conductor bundle.
- **geometry** (Geometry) - Geometry object representing line physical layout.
- **length** (float) - Length of the transmission line in km.
- **zbase** (float) **-** Base impedance calculated from bus1 voltage and system base power.
- **series_impedance** (complex) **-** Complex series impedance in per-unit.
- **shunt_admittance** (complex) **-** Complex shunt admittance in per-uni
- **yprim** (np.ndarray) **-** 2x2 primitive admittance matrix.

Methods:

- **__init__**(self, name: str, bus1: Bus, bus2: Bus, bundle: Bundle, geometry: Geometry, length: float)
  Initializes a TransmissionLine object with name, buses, bundle, geometry, and length. Calculates zbase, series impedance, shunt admittance, and primitive admittance matrix.
- **calculate_series_impedance**(self)
  Calculates and returns the per-unit series impedance using physical and electrical properties.
- **calculate_shunt_admittance**(self)
  Calculates and returns the per-unit shunt admittance of the line.
- **calc_yprim**(self)
  Computes and returns the 2x2 primitive admittance matrix for the transmission line.
- **__str__**(self)
  Returns a string summary of the transmission line attributes and calculated parameters.

Transmission Line Subclasses:

Geometry Class

Attributes:

- **name** (str) – Name of the geometry object.

- **xa** (float) – x-coordinate of conductor A.
- **ya** (float) – y-coordinate of conductor A.
- **xb** (float) – x-coordinate of conductor B.
- **yb** (float) – y-coordinate of conductor B.
- **xc** (float) – x-coordinate of conductor C.
- **yc** (float) – y-coordinate of conductor C.
- **DEQ** (float) – Equivalent geometric distance calculated from conductor positions. Computed during initialization.

Methods:

- **init**(self, name: str, xa: float, ya: float, xb: float, yb: float, xc: float, yc: float)
  Constructor that initializes a Geometry object with three conductor coordinates and computes the DEQ value.
- **calcDEQ**(self)
  Calculates the equivalent distance (DEQ) between three conductors using geometric mean. Called during object initialization.

Conductor Class

Attributes:

- **name** (str) – Name of the conductor type.
- **diam** (float) – Physical diameter of the conductor in inches
- **GMR** (float) – Geometric Mean Radius of the conductor in feet.
- **resistance** (float) – Electrical resistance of the conductor in ohms per mile.
- **ampacity** (float) – Maximum allowable current (ampacity) the conductor can carry, in amperes.

Methods:

- **init**(self, name: str, diam: float, GMR: float, resistance: float, ampacity: float)
  Constructor that initializes a Conductor object with geometric and electrical parameters including name, diameter, GMR, resistance, and ampacity.

Bundle Class

Attributes:

- **name** (str) – Name of the conductor bundle.
- **num_conductors** (float) – Number of conductors in the bundle.

- **spacing** (float) – Spacing between conductors in feet.
- **conductor** (Conductor) – Conductor object representing the physical conductor used in the bundle.
- **DSC** (float) – Equivalent conductor spacing used for capacitance calculations, in feet. Calculated internally.
- **DSL** (float) – Equivalent conductor spacing used for inductance calculations, in feet. Calculated internally.

Methods:

- **init**(self, name: str, num_conductors: float, spacing: float, conductor: Conductor)
  Constructor that initializes a Bundle object with the number of conductors, spacing, and associated Conductor. Also triggers calculation of DSL and DSC.
- **calc_DSC**(self)
  Calculates the DSC value based on the number of conductors and spacing. Uses empirical formulas for different bundling configurations.
- **calc_DSL**(self)
  Calculates the DSL value using GMR and spacing. Uses appropriate formula depending on the number of bundled conductors.

Example Usage:

```
circuit1=Circuit("Circuit1")

#Adding buses
circuit1.add_bus("bus1",20,bus_type="Slack_Bus")
circuit1.add_bus("bus2",230)
circuit1.add_bus("bus3",230)
circuit1.add_bus("bus4",230)
circuit1.add_bus("bus5",230)
circuit1.add_bus("bus6",230)
circuit1.add_bus("bus7",18,bus_type="PV_Bus")

#Transmission Line sub-classes
conductor1=Conductor("Partridge",0.642,0.0217,0.385,460)
bundle1=Bundle("Bundle1",2,1.5,conductor1)
geometry1=Geometry("Geometry1",0,0,9.75*2,0,9.75*4,0)

#Adding Transmission Lines
circuit1.add_transmission_lines("Line1","bus2","bus4",bundle1,geometry1,10)
circuit1.add_transmission_lines("line2","bus2","bus3",bundle1,geometry1,25)
circuit1.add_transmission_lines("line3","bus3","bus5",bundle1,geometry1,20)
circuit1.add_transmission_lines("Line4","bus4","bus6",bundle1,geometry1,20)
circuit1.add_transmission_lines("Line5","bus5","bus6",bundle1,geometry1,10)
circuit1.add_transmission_lines("Line6","bus4","bus5",bundle1,geometry1,35)
```

```
#Adding Transformers
circuit1.add_transformer("Tx1","bus1","bus2",125,8.5,10)
circuit1.add_transformer("Tx2","bus6","bus7",200,10.5,12)
```

Appendix:

Geometric mean distance between conductors, used in Geometry class

$$d_{AB} = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$$

$$d_{AC} = \sqrt{(x_a - x_c)^2 + (y_a - y_c)^2}$$

$$d_{BC} = \sqrt{(x_b - x_c)^2 + (y_b - y_c)^2}$$

$$DEQ = (d_{AB} \cdot d_{AC} \cdot d_{BC})^{1/3}$$

DSL (Spacing for Inductance Calculations), used in Bundle class

$$\text{DSL} = \begin{cases} \text{GMR}, & \text{if } n = 1 \\ (\text{GMR} \cdot \text{spacing})^{1/2}, & \text{if } n = 2 \\ (\text{GMR} \cdot \text{spacing}^2)^{1/3}, & \text{if } n = 3 \\ (\text{GMR} \cdot \text{spacing}^3)^{1/4}, & \text{if } n = 4 \end{cases}$$

DSC (Spacing for Capacitance Calculations), used in Bundle class

$$r = \frac{\text{diameter}}{24}$$

$$\text{DSC} = \begin{cases} r, & \text{if } n = 1 \\ (r \cdot \text{spacing})^{1/2}, & \text{if } n = 2 \\ (r \cdot \text{spacing}^2)^{1/3}, & \text{if } n = 3 \\ 1.091 \cdot (r \cdot \text{spacing}^3)^{1/4}, & \text{if } n = 4 \end{cases}$$

Base Impedance, used throughout classes for per-unitization

$$Z_{base} = \frac{V_{base}^2}{S_{base}}$$

Per-unit Series Resistance, Reactance, and Impedance, used in Transmission Line Class

$$R_a = \frac{R_{conductor}}{n_{cond}} \cdot \frac{L}{Z_{base}}$$

$$X_a = \frac{2\pi f \cdot 2 \times 10^{-7} \cdot \log\left(\frac{D_{eq}}{D_{SL}}\right) \cdot 1609 \cdot L}{Z_{base}}$$

$$Z_{series} = R_a + jX_a$$

Per-unit Shunt Admittance, used in Transmission Line Class

$$Y = \left(\frac{2\pi f \cdot 1609 \cdot 2\pi \cdot 8.854 \times 10^{-12}}{\log\left(\frac{D_{eq}}{D_{SC}}\right)} \cdot L\right) \cdot Z_{base}$$

Primitive Admittance Matrix of T-Line, used in Transmission Line Class

$$Y_{prim} = \begin{bmatrix} Y_{series} + \frac{Y_{shunt}}{2} & -Y_{series} \\ -Y_{series} & Y_{series} + \frac{Y_{shunt}}{2} \end{bmatrix}$$

Circuit Class

Overview

The circuit class is the backbone of the simulator. It contains buses, transmission lines, generators, loads, and transformers. All of these components are connected together via the circuit class. This class also contains methods that assist in large scale calculations for the entire circuit.

Attributes:

- **name** (str) – Name of the circuit.
- **buses**(Dict[str, Bus])- A dictionary with bus objects as the value and the bus names as the keys.
- **transformers**(Dict[str, Transformer])- A dictionary with transformer objects as the value and their name as the key.

- **transmission_lines**(Dict[str, Transmission_Line])- A dictionary with transmission line objects as the value and their name as the key.
- **generators**(DIct[str, generators])- A dictionary that contains generator objects as the values and their names as the key.
- **loads**(Dict[str, load])- A dictionary with load objects as the value and their name as the key.
- **Ybus**- An array that contains the y-bus admittance matrix for the circuit.

Methods:

- **__init__()** – Initializes a circuit object.
- **add_bus**(self, name:str, base_kv:float, vpu:float=1, delta:float=0 , bus_type: str = "PQ_Bus") – Initializes a new bus and adds it to the buses dict. Vpu, delta, and bus_type all have default values. The method also checks for repeated bus names.
- **add_transformer**(self, name:str, bus1:str, bus2:str, power_rating:float, impedance_percent :float, x_over_r_ratio:float) – Method initializes and adds a new transformer object into the circuit. The method also checks for duplicate transformer names and if the connected buses exist before adding transformer.
- **add_transmission_lines**(self, name:str, bus1:str, bus2:str, bundle:Bundle, geometry:Geometry, length:float) – This method adds a new transmission line between two buses into the system. Method checks for duplicate names as well as if the buses exist. Requires subclases bundle and geometry to create new transmission line.
- **add_generator**(self, name:str, voltage_setpoint:float, mw_setpoint:float, gen_bus_name:str, sub_trans:float) – Method is used to add a new generator object to the system. Method checks to make sure that the bus being this generator is added to is a PV bus or a slack bus.
- **add_load**(self, name:str, real_power:float, reactive_power:float, load_bus_name:str) – Method is used to add new loads to the system. Method checks to make sure that the bus is a PQ bus before being added to the system.
- **calc_y_admit**(self) – Method that calculates the Y-Bus admittance matrix. This uses the primitive admittance matrices from transformers and transmission line classes to create the overall admittance matrix.
- **print_ybus**(self)- Allows the user to print and inspect the admittance matrix.

Example Usage:

```
circuit1=Circuit("Circuit1")

#Adding buses
 circuit1.add_bus("bus1",20,bus_type="Slack_Bus")
 circuit1.add_bus("bus2",230)
 circuit1.add_bus("bus3",230)
 circuit1.add_bus("bus4",230)
 circuit1.add_bus("bus5",230)
 circuit1.add_bus("bus6",230)
 circuit1.add_bus("bus7",18,bus_type="PV_Bus")

#Transmission Line sub-classes
 conductor1=Conductor("Partridge",0.642,0.0217,0.385,460)
 bundle1=Bundle("Bundle1",2,1.5,conductor1)
 geometry1=Geometry("Geometry1",0,0,9.75*2,0,9.75*4,0)

#Adding Transmission Lines
 circuit1.add_transmission_lines("Line1","bus2","bus4",bundle1,geometry1,10)
 circuit1.add_transmission_lines("line2","bus2","bus3",bundle1,geometry1,25)
 circuit1.add_transmission_lines("line3","bus3","bus5",bundle1,geometry1,20)
 circuit1.add_transmission_lines("Line4","bus4","bus6",bundle1,geometry1,20)
 circuit1.add_transmission_lines("Line5","bus5","bus6",bundle1,geometry1,10)
 circuit1.add_transmission_lines("Line6","bus4","bus5",bundle1,geometry1,35)

#Adding Transformers
 circuit1.add_transformer("Tx1","bus1","bus2",125,8.5,10)
 circuit1.add_transformer("Tx2","bus6","bus7",200,10.5,12)

circuit1.calc_y_admit()
```

Appendix:

$$Y_{\text{bus}}(k,k) = \sum \text{pu admittances connected to bus } k$$

$$Y_{\text{bus}}(k,m) = -\sum \text{pu admittances connecting bus } k \text{ to bus } m, \quad \text{where } k \neq m$$

Figure 1: Formula for Constructing Ybus Admittance Matrix

Index	bus1	bus2	bus3	bus4	bus5	bus6	bus7
bus1	1.46329-14.63290j	-1.46329+14.63290j	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j
bus2	-1.46329+14.63290j	37.79137-127.06982j	-10.37945+32.14339j	-25.94863+80.35848j	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j
bus3	0.00000+0.00000j	-10.37945+32.14339j	23.35376-72.23912j	0.00000+0.00000j	-12.97431+40.17924j	0.00000+0.00000j	0.00000+0.00000j
bus4	0.00000+0.00000j	-25.94863+80.35848j	0.00000+0.00000j	46.33683-143.37666j	-7.41389+22.95957j	-12.97431+40.17924j	0.00000+0.00000j
bus5	0.00000+0.00000j	0.00000+0.00000j	-12.97431+40.17924j	-7.41389+22.95957j	46.33683-143.37666j	-25.94863+80.35848j	0.00000+0.00000j
bus6	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j	-12.97431+40.17924j	-25.94863+80.35848j	40.50476-139.46387j	-1.58182+18.98182j
bus7	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j	0.00000+0.00000j	-1.58182+18.98182j	1.58182-18.98182j

Figure 2: Example of Calculated Ybus Admittance Matrix.

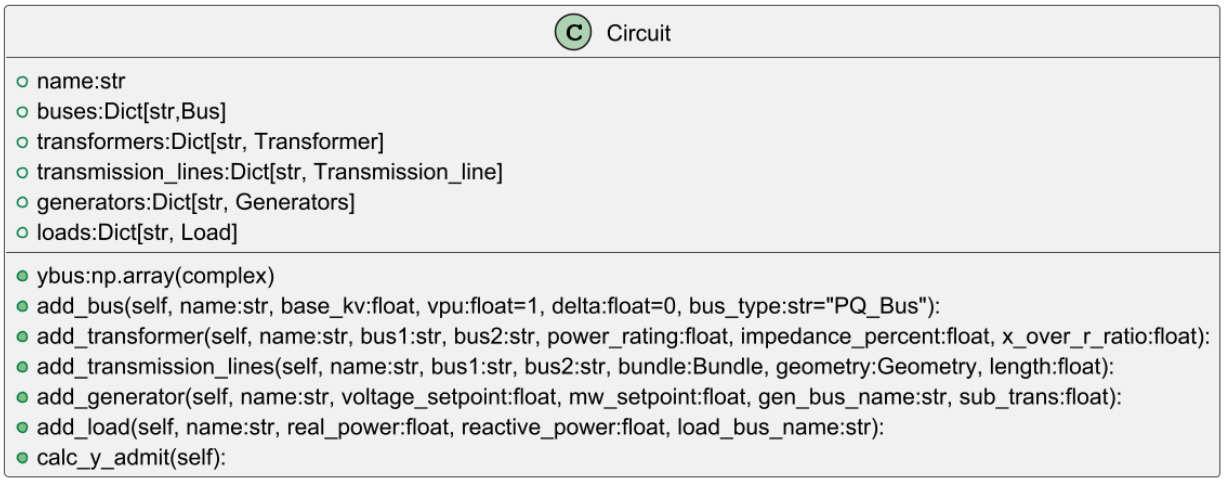


Figure 3: Class Diagram

Powerflow Class

Overview

The Powerflow Class takes data from the Circuit class and uses it to compute power injection and power mismatch for each bus. This data is then used for solving the powerflow model of the system using the Newton-Raphson algorithm.

Class Diagram

Class Implementation

Attributes:

- **circuit -** Circuit class object that contains data on buses, loads, and generators

Methods:

- **__init__**(self, circuit: Circuit)
  Initializes Powerflow object to compute data based on the Circuit object passed as a parameter
- **flat_start**(self)
  Sets the voltage magnitudes at each bus to 1 and voltage angles at each bus to 0
- **calc_PQ**(self)
  Calculates the injected power at each bus. Returns a tuple of lists for real and imaginary power
- **calc_mismatch**(self, p_injected, q_injected)
  When passed injected power at each bus, calculates the mismatch between expected power values. Returns a single list of concatenated real and reactive power mismatches.

Appendix:

Injected Power at given Bus

$$P_i^{\text{calc}} = \sum_{j=1}^{n} V_i V_j \left(G_{ij} \cos(\delta_i - \delta_j) + B_{ij} \sin(\delta_i - \delta_j) \right)$$

$$Q_i^{\text{calc}} = \sum_{j=1}^{n} V_i V_j \left(G_{ij} \sin(\delta_i - \delta_j) - B_{ij} \cos(\delta_i - \delta_j) \right)$$

Where:

- G_{ij} and B_{ij} are the conductance and susceptance from the bus admittance matrix Y_{bus}
- V_j, δ_j are the voltage magnitude and angle at bus j
- P_i^{spec}: Specified real power (generation - load)
- Q_i^{spec}: Specified reactive power (generation - load)
- V_i: Voltage magnitude at bus i
- δ_i: Voltage angle at bus i

Power Mismatch at given Bus

$$\Delta P_i = P_i^{\text{spec}} - P_i^{\text{calc}}$$

$$\Delta Q_i = Q_i^{\text{spec}} - Q_i^{\text{calc}}$$

Jacobian Class

Overview

In the context of power flow analysis using the Newton-Raphson (NR) algorithm, the Jacobian matrix represents the partial derivatives of power mismatches with respect to voltage magnitudes and angles. It serves as the sensitivity matrix that helps determine how changes in bus voltages affect real (P) and reactive (Q) power flows.

Attributes:

- **circuit** – The Jacobian class takes a circuit object as an input.
- **Jacobian**- Stores the final Jacobian Matrix.

Methods:

- **calcJ1**(self)- This method calculates the untrimmed top-left quadrant of the Jacobian matrix.
- **calcJ2**(self)- This method calculates the untrimmed top-right quadrant of the final Jacobian matrix.
- **calcJ3**(self)- This method calculates the untrimmed bottom-left quadrant of the Jacobian matrix.
- **calcJ4**(self)- This method calculates the untrimmed bottom-right quadrant of the final Jacobian matrix.
- **calc_jacobian**(self) – This method calls previous methods to calculate Jacobian in pieces, then depending on the PV and Slack bus removes rows and columns to form the trimmed jacobian. This function returns the trimmed jacobian matrix as an np array.
- **print_jacobian**(self) – This method allows the user to print the trimmed Jacobian matrix as a pandas data frame.

Theory:

In Newton-Raphson power flow analysis, the Slack and PV buses removed from the Jacobian because their values are fixed. The Slack bus has a fixed voltage magnitude and

angle, while PV buses have a fixed voltage magnitude. Since these values are not updated during iterations, excluding them from the Jacobian reduces computational effort and focuses the solution on the actual unknowns.

Appendix:

$n \neq k$

$$J1_{kn} = \frac{\partial P_k}{\partial \delta_n} = V_k Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn})$$

$$J2_{kn} = \frac{\partial P_k}{\partial V_n} = V_k Y_{kn} \cos(\delta_k - \delta_n - \theta_{kn})$$

$$J3_{kn} = \frac{\partial Q_k}{\partial \delta_n} = -V_k Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn})$$

$$J4_{kn} = \frac{\partial Q_k}{\partial V_n} = V_k Y_{kn} \sin(\delta_k - \delta_n - \theta_{kn})$$

$n = k$

$$J1_{kk} = \frac{\partial P_k}{\partial \delta_k} = -V_k \sum_{\substack{n=1 \\ n \neq k}}^{N} Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn})$$

$$J2_{kk} = \frac{\partial P_k}{\partial V_k} = V_k Y_{kk} \cos\theta_{kk} + \sum_{n=1}^{N} Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn})$$

$$J3_{kk} = \frac{\partial Q_k}{\partial \delta_k} = V_k \sum_{\substack{n=1 \\ n \neq k}}^{N} Y_{kn} V_n \cos(\delta_k - \delta_n - \theta_{kn})$$

$$J4_{kk} = \frac{\partial Q_k}{\partial V_k} = -V_k Y_{kk} \sin\theta_{kk} + \sum_{n=1}^{N} Y_{kn} V_n \sin(\delta_k - \delta_n - \theta_{kn})$$

Figure 1: Formula For constructing the un-trimmed matrix.

Index	0	1	2	3	4	5	6	7	8	9	10
0	127.13478	-32.14339	-80.35848	0.00000	0.00000	0.00000	-0.00000	-10.37945	-25.94863	0.00000	0.00000
1	-32.14339	72.32264	0.00000	-40.17924	0.00000	0.00000	-10.37945	-0.00000	0.00000	-12.97431	0.00000
2	-80.35848	0.00000	143.49729	-22.95957	-40.17924	0.00000	-25.94863	0.00000	-0.00000	-7.41389	-12.97431
3	0.00000	-40.17924	-22.95957	143.49729	-80.35848	0.00000	0.00000	-12.97431	-7.41389	-0.00000	-25.94863
4	0.00000	0.00000	-40.17924	-80.35848	139.51955	-18.98182	0.00000	0.00000	-12.97431	-25.94863	-0.00000
5	0.00000	0.00000	0.00000	0.00000	-18.98182	18.98182	0.00000	0.00000	0.00000	0.00000	-1.58182
6	-37.79137	10.37945	25.94863	-0.00000	-0.00000	-0.00000	-8.25386	-32.14339	-80.35848	0.00000	0.00000
7	10.37945	-23.35376	-0.00000	12.97431	-0.00000	-0.00000	-32.14339	-6.03298	0.00000	-40.17924	0.00000
8	25.94863	-0.00000	-46.33683	7.41389	12.97431	-0.00000	-80.35848	0.00000	-17.29571	-22.95957	-40.17924
9	-0.00000	12.97431	7.41389	-46.33683	25.94863	-0.00000	0.00000	-40.17924	-22.95957	-17.29571	-80.35848
10	-0.00000	-0.00000	12.97431	25.94863	-40.50476	1.58182	0.00000	0.00000	-40.17924	-80.35848	-7.76479

Figure 2: Sample output of print_jacobian method

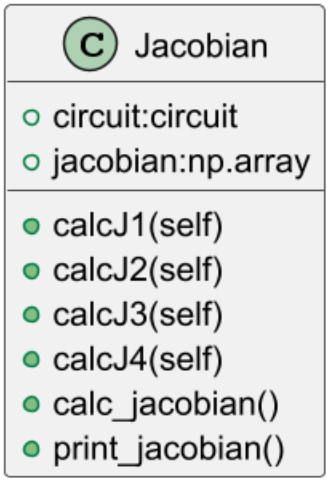


Figure 3: Jacobian Class Diagram

Newton_Raphson Class

Attributes:

- **circuit** (Circuit object)– Power system network with necessary data
- **tol** (float = 0.001) – Convergence tolerance for pu mismatches
- **max_iter** (int = 50) – Maximum iterations for NR algorithm
- **buses** (dict) – Reference to circuit.buses for quick access
- **powerflow** (Powerflow object) – Helper class for power flow calculations (flat start, mismatch computation)
- **p_inj, q_inj** (numpy.ndarray) - Real/reactive power injection vectors (updated each iteration)

Methods:

- **__init__(**self, circuit: Circuit, tol: float, max_iter: int)
  Initializes Newton_Raphson object and stores circuit data, tolerance, and max iterations. Calculates initial power injections
- **solve**(self)
  Executes Newton-Raphson Algorithm by calculating mismatches, checking against tolerance, constructing Jacobian, and updating bus voltages for each iteration.
- **update_voltages**(self, delta_x: np.ndarray)
  Applies voltage corrections for magnitude and angle to each bu
- **format_mismatch_dataframe**(self, mismatch_vector: np.ndarray) -> pd.DataFrame
  Formats mismatches into a labeled dataframe for debugging
- **format_delta_x(**self, delta_x: np.ndarray) -> pd.DataFrame
  Formats voltage differences into a dataframe for debugging

Appendix

Newton-Raphson Algorithm

1. Initial Setup
    a. Get Load/Generator data for each bus
    b. Set bus voltages with flat start
2. Power Mismatch Calculation
    a. See Powerflow Class for details
3. Jacobian Matrix Construction
    a. See Jacobian Class for details
4. Solving for bus voltage differences given Jacobian J and Power Mismatches

$$J \cdot \begin{bmatrix} \Delta\delta \\ \Delta V \end{bmatrix} = \begin{bmatrix} \Delta P \\ \Delta Q \end{bmatrix}$$

5. Update bus voltages

$$\delta^{(k+1)} = \delta^{(k)} + \Delta\delta$$

$$V^{(k+1)} = V^{(k)} + \Delta V$$

6. Check for convergence, repeat if max mismatch > tolerance

Symmetrical Fault Class

Overview

A symmetrical fault (also known as a balanced fault) is a type of electrical fault in a power system where all three phases (R, Y, B or A, B, C) are equally affected. It usually involves a short circuit between all three phases or between all three phases and ground, and the magnitude and phase angle of the currents in each phase are the same. This part of the simulator allows users to see the bus voltages and the fault current while selecting what bus a symmetrical fault happens

---

Attributes:

- **circuit** – The Symmetrical Fault class takes a circuit object as an input.
- **bus_fault**:str- The name of the bus where the fault occurs.
- **Zbus**:str- The inverted ybus after generator sub-transient reactances are accounted for.
- **fault_current**:float – Stores the calculated value of the fault current.
- **Fault_voltages**:np.array- The fault voltages at each bus are stored in an array.

Methods:

- **Init**(self, circuit, bus_fault)- Initializes the class.
- **Calc_fault**(self) – This method uses the circuit object passed in to alter the y-bus by adding the generator sub-transient reactance. Then using some theory explained below uses the updated ybus matrix to calculate fault voltages and the fault current.
- **print_fault_voltages**(self) – Allows the user to print the fault voltages as a pandas data frame.
- **print_zbus**(self) – Allows the user to print the z-bus matrix as a pandas data frame.

- **print_ybus**(self)- Allows the user to print the updated y-bus admittance matrix as a data frame.

Theory

The calculation of symmetrical faults is based on representing the power system using the per-unit system and applying Thevenin's theorem. In this method, all sources are replaced by their internal impedances, and the system is simplified to its Thevenin equivalent as seen from the fault location. Since a symmetrical fault is balanced, only the positive sequence network is considered.

$I_{fault} = \frac{V_{pre-fault}}{Z_{n,n}}$ - Equation for the fault current when the fault is at bus n.

$E_K = \left(1 - \frac{Z_{nk}}{Z_{nn}}\right) * V_F$- Equation for the fault voltage at bus k when there is a fault at bus n.

To calculate the z-bus, it is a matrix inversion operation on the updated y-bus. The notation Z_{nk} in the above equations is the index in the z-bus.

Appendix

Index	bus1	bus2	bus3	bus4	bus5	bus6	bus7
bus1	0.00174+0.08373j	0.00067+0.06317j	-0.00001+0.06065j	-0.00000+0.06068j	-0.00059+0.05852j	-0.00108+0.05676j	-0.00175+0.03948j
bus2	0.00067+0.06317j	0.00462+0.09876j	0.00341+0.09485j	0.00341+0.09489j	0.00237+0.09155j	0.00151+0.08882j	-0.00051+0.06184j
bus3	-0.00001+0.06065j	0.00341+0.09485j	0.00800+0.10907j	0.00338+0.09479j	0.00432+0.09771j	0.00270+0.09274j	0.00025+0.06459j
bus4	-0.00000+0.06068j	0.00341+0.09489j	0.00338+0.09479j	0.00524+0.10056j	0.00329+0.09453j	0.00264+0.09255j	0.00021+0.06445j
bus5	-0.00059+0.05852j	0.00237+0.09155j	0.00432+0.09771j	0.00329+0.09453j	0.00581+0.10246j	0.00359+0.09570j	0.00081+0.06667j
bus6	-0.00108+0.05676j	0.00151+0.08882j	0.00270+0.09274j	0.00264+0.09255j	0.00359+0.09570j	0.00425+0.09796j	0.00124+0.06825j
bus7	-0.00175+0.03948j	-0.00051+0.06184j	0.00025+0.06459j	0.00021+0.06445j	0.00081+0.06667j	0.00124+0.06825j	0.00177+0.08400j

Figure 1: Sample Z-bus print

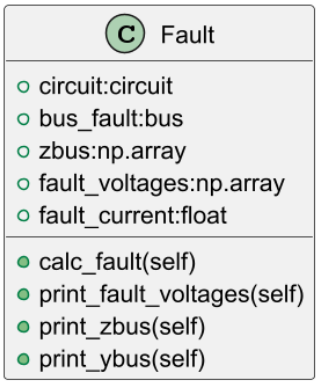


Figure 2: Class Diagram

Project #3:

Case Information Graphical User Interface

Project Overview

My addition to the circuit simulator is a graphical user interface that shows all case information, similar in style to what can be found in the power world simulator. This user interface allows the user to see all data from the simulator in one convenient location. The user interface is easily navigable, which makes it a great user-friendly tool to add to the simulator.

From the GUI, a user has three categories of information available: Equipment data, Solution Details, and Fault Study data. From the equipment data section, the user can select to see information about the buses, generators, transfromers, loads, and transmission lines. From the Solution details category, the user can find the y-bus, the jacobian matrix, and the initial power flow details. The Fault Study Section contains the z-bus, the updated y-bus, and the fault voltages and currents.

Attributes:

- Master
- Circuit:circuit
- Bus_fault:str

Methods:

- **Init**(self, master, circuit, bus_fault)- Initializes the class. Uses the circuit object to print and calculate all needed values. Also, this method sets up the initial framework of the GUI.
- **show_equioment_info**(self, event)- This method is used when the user clicks on a node in the tree. The method formats the data from the calculations done inside of the circuit class in a way that is easily readable to the user.
- **display_dataframe**(self, df: pd.DataFrame)- This method is similar to show_equipment_info, but is used instead to easily display the data that is already in dataframes, such as the y-bus and Jacobian matrices.

Appendix

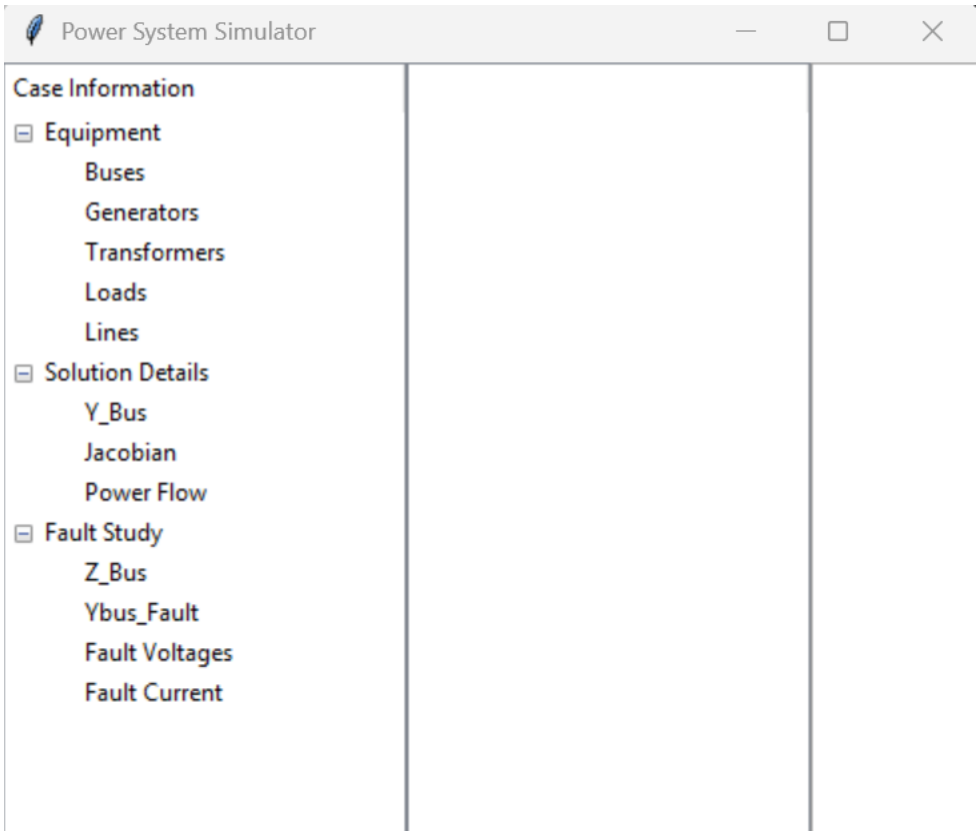


Figure 1: Initialization of the GUI

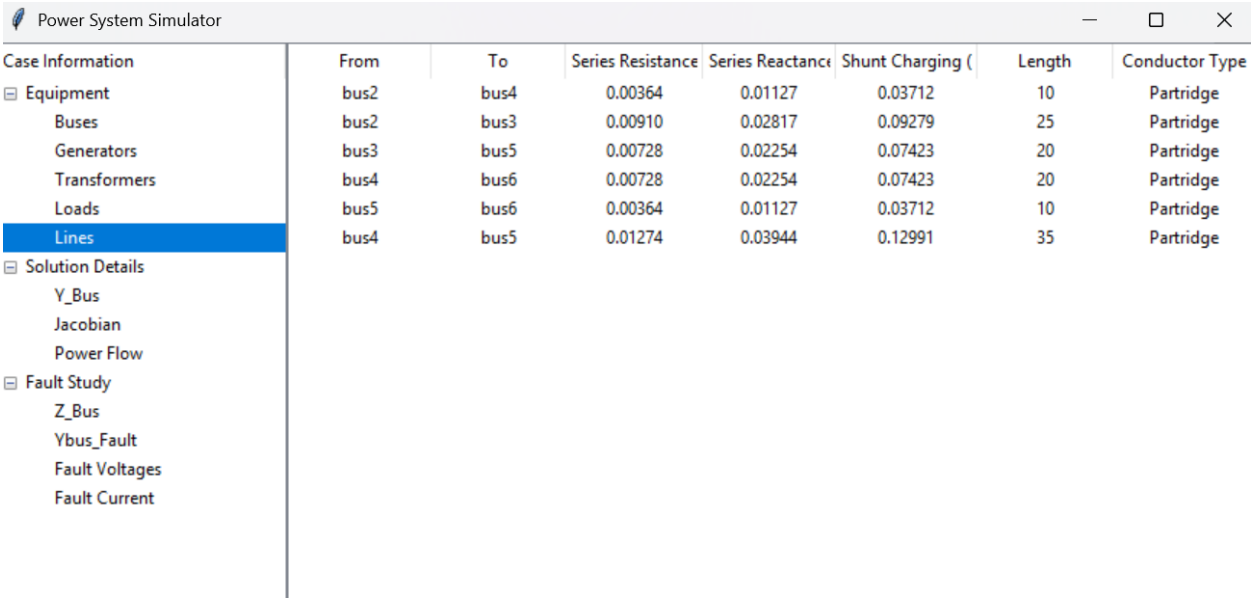


Figure 2: Reading Transmission Line Data

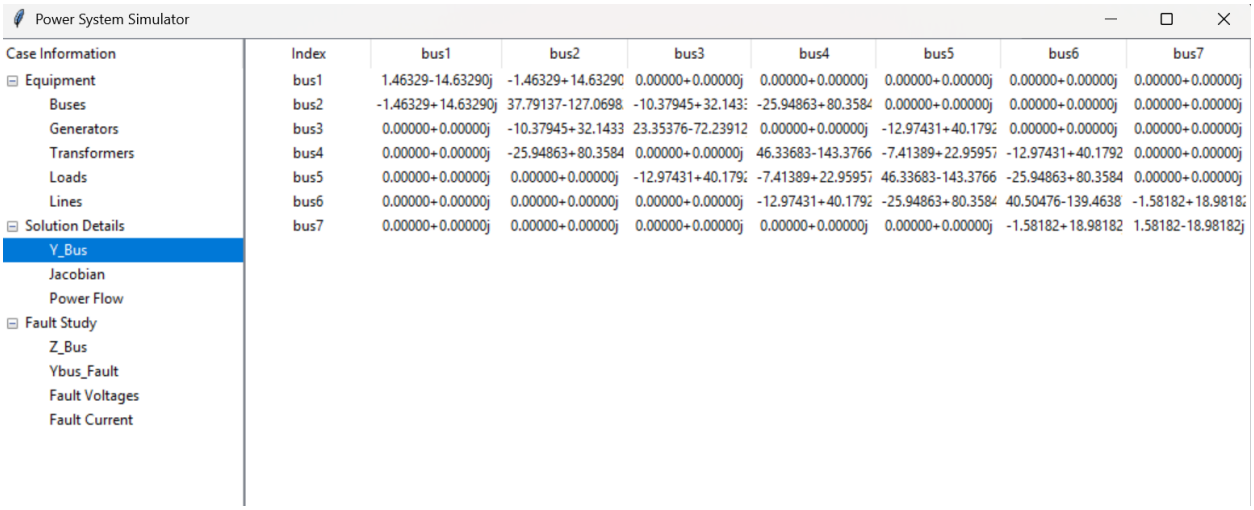


Figure 3: Reading Y-Bus data

Example Usage for Entire Simulator

The example code below is used to create a 7 bus system with 2 generators and 4 loads. The generators are on bus 1 and bus 7, with the loads being on bus 3, 4, and 5. Shown below is a graphical view of the 7 bus system. The code also contains the commands to run the graphical user interface, implemented as project 3.

```
#import all member classes


import numpy as np
import pandas as pd
from Bus import Bus
from Geometry import Geometry
from Conductor import Conductor
from Transformer import Transformer
from Bundle import Bundle
from TransmissionLine import TransmissionLine
from Geometry import Geometry
from Load import Load
from Generator import Generator
from circuit import Circuit
from Jacobian import Jacobian
from Symmetrical_Faults import Fault
import tkinter as tk
from GUI2 import PowerSystemGUI2


#Defining new circuit object
circuit1 = Circuit("Circuit1")

# Adding buses
circuit1.add_bus("bus1", 20, bus_type="Slack_Bus")
circuit1.add_bus("bus2", 230)
circuit1.add_bus("bus3", 230)
circuit1.add_bus("bus4", 230)
circuit1.add_bus("bus5", 230)
circuit1.add_bus("bus6", 230)
circuit1.add_bus("bus7", 18, bus_type="PV_Bus")

# Transmission Line sub-classes
conductor1 = Conductor("Partridge", 0.642, 0.0217, 0.385, 460)
bundle1 = Bundle("Bundle1", 2, 1.5, conductor1)
geometry1 = Geometry("Geometry1", 0, 0, 9.75 * 2, 0, 9.75 * 4, 0)

# Adding Transmission Lines
```

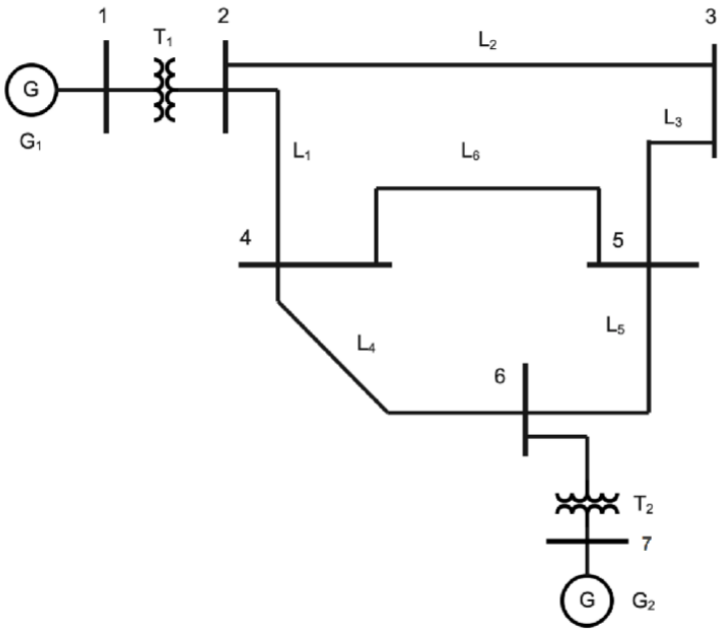
```
circuit1.add_transmission_lines("Line1", "bus2", "bus4", bundle1, geometry1,
10)
circuit1.add_transmission_lines("line2", "bus2", "bus3", bundle1, geometry1,
25)
circuit1.add_transmission_lines("line3", "bus3", "bus5", bundle1, geometry1,
20)
circuit1.add_transmission_lines("Line4", "bus4", "bus6", bundle1, geometry1,
20)
circuit1.add_transmission_lines("Line5", "bus5", "bus6", bundle1, geometry1,
10)
circuit1.add_transmission_lines("Line6", "bus4", "bus5", bundle1, geometry1,
35)

# Adding Transformers
circuit1.add_transformer("Tx1", "bus1", "bus2", 125, 8.5, 10)
circuit1.add_transformer("Tx2", "bus6", "bus7", 200, 10.5, 12)

#Adding Generators
circuit1.add_generator("Gen1", 20, 100, "bus1", 0.12)
circuit1.add_generator("Gen2", 18, 100, "bus7", 0.12)

# Adding Loads
circuit1.add_load("Load1", 0, 0, "bus2")
circuit1.add_load("Load2", 110, 50, "bus3")
circuit1.add_load("Load3", 100, 70, "bus4")
circuit1.add_load("Load4", 100, 65, "bus5")
circuit1.add_load("Load5", 0, 0, "bus6")

#Adding and running the GUI
root = tk.Tk()
power = PowerSystemGUI2(root, circuit1,"bus1")
root.mainloop()
```



The 7 bus system created by the code example above.